Foundational Security Concept: CIA Triad

1. Configure File Permissions for Confidentiality and Integrity

In this step, I will use Linux file permissions to protect sensitive data.

Create the Environment

First, I will create and enter a project directory to keep my work organized.

Terminal:

```
┌──(kali㉿kali)-[~]
└─$ mkdir -p ~/project && cd ~/project
```

- **`mkdir -p ~/project`**: Creates the directory if it doesn't exist.
- **`cd ~/project`**: Moves the terminal session into that directory.

Create a Sensitive File

Terminal:

```
┌──(kali㉿kali)-[~/project]
└─$ echo "Top Secret Details" > confidential_data.txt
```

- **`echo "..." >`**: Writes the specified text into a file.
- **`ls -l`**: Lists the file with detailed permissions. By default, files often show `-rw-rw-r--`, meaning others can read the file, which violates **Confidentiality**.

Enforce Confidentiality with `chmod`

Terminal:

```
┌──(kali㉿kali)-[~/project]
└─$ chmod 600 confidential_data.txt
```

- **`chmod`**: This command changes the "mode" or permissions of a file.
- **The numeric code `600`**:
  - The first digit (**6**) gives the **Owner** (kali) Read (4) and Write (2) permissions ($4+2=6$).
  - The second digit (**0**) removes all access for the **Group**.
  - The third digit (**0**) removes all access for **Others**.
  - **Result**: Only the user `kali` can access the data.

Enforce Integrity with `chmod`

To protect the file from accidental changes (Integrity), I will make it read-only.

Terminal:

```
┌──(kali㉿kali)-[~/project]
└─$ chmod 400 confidential_data.txt
┌──(kali㉿kali)-[~/project]
└─$ echo "This should not be added." >> confidential_data.txt
zsh: permission denied: confidential_data.txt
```

- **`chmod 400`**: Changes the permission to **Read-Only** (4) for the owner and no access for anyone else.
- **Result**: Any attempt to modify or append text (>>) results in a "Permission denied" error.

2. Demonstrate File Integrity with Hashing

While permissions stop unauthorized edits, **cryptographic hashing** detects if a file has been tampered with or corrupted.

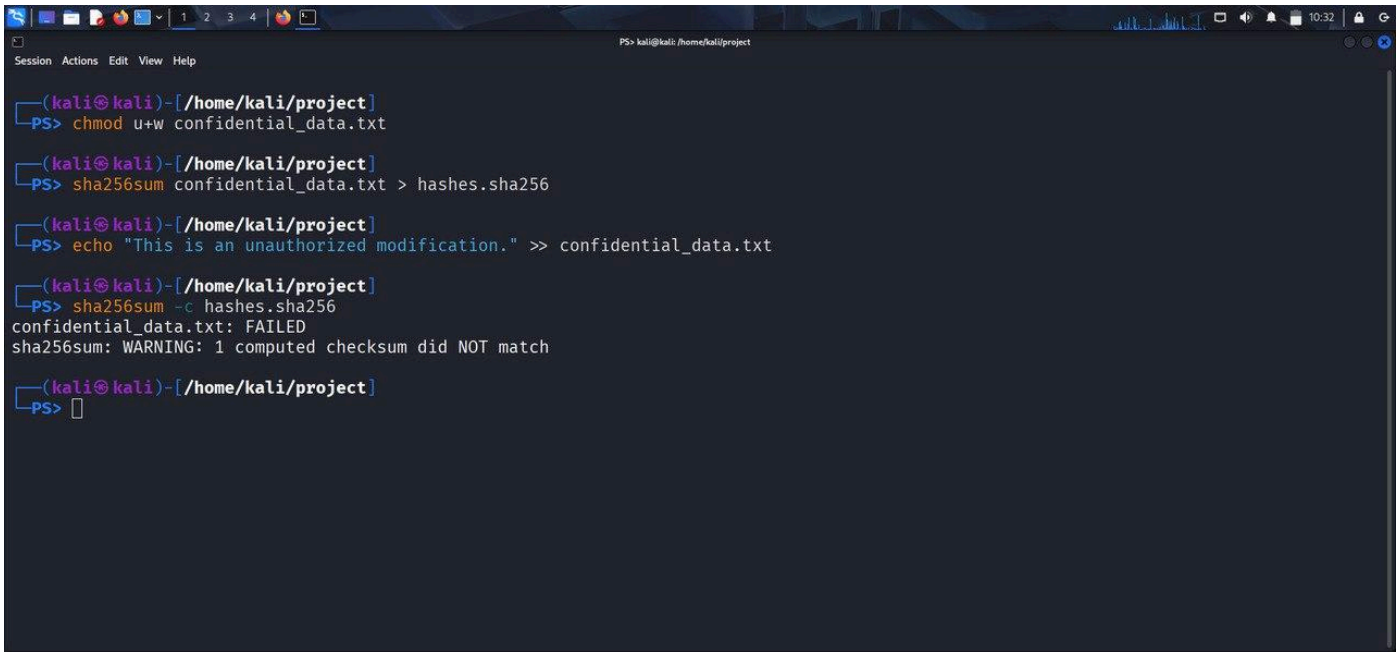
Generate and Verify Hashes

Terminal:

```
┌──(kali㉿kali)-[~/project]
└─$ chmod u+w confidential_data.txt
┌──(kali㉿kali)-[~/project]
└─$ sha256sum confidential_data.txt > hashes.sha256
┌──(kali㉿kali)-[~/project]
└─$ echo "This is an unauthorized modification." >> confidential_data.txt
┌──(kali㉿kali)-[~/project]
└─$ sha256sum -c hashes.sha256
```

```
confidential_data.txt: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
```

- **`chmod u+w`**: Re-enables **Write** access for the **User** so we can simulate a change.
- **`sha256sum`**: Generates a unique 64-character "fingerprint" of the file content.
- **`sha256sum -c or --check`**: The "check" flag compares the current file against the previously saved hash. If even one bit changed, it reports **FAILED**.



3. Simulate Service Availability and Monitoring

Availability ensures that systems are functional and reachable by authorized users when needed.

Start and Monitor a Service

Terminal:

```
┌──(kali㉿kali)-[~/project]
└─$ python3 -m http.server 8000 &
[1] 45231
┌──(kali㉿kali)-[~/project]
└─$ curl http://localhost:8000/confidential_data.txt
```

- **`python3 -m http.server 8000 &`**: Starts a web server on port 8000 in the **background** (indicated by the `&`).
- **`curl`**: A tool to transfer data from a server; if it returns my text, then the service is **Available**.
- **`kill 45231`**: Stops the process using its Process ID (PID) to simulate a service crash.

Automate with a Monitoring Script

Finally, let's create a simple monitoring script to automate this check. Create a new file named **monitor.py** using **nano**.

```
nano monitors.py
```

Enter the following script into the editor. This script uses **curl** to check the service. If it gets a successful response (**200 OK**), it reports the service is "UP"; otherwise, it reports "DOWN".

```
#!/usr/bin/env python3


import requests

import sys

from datetime import datetime


URL = "http://localhost:8000"

TIMEOUT = 5


def check_service():

    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")


    try:

        response = requests.head(URL, timeout=TIMEOUT)


        if response.status_code == 200:

            print(f"[{timestamp}] Service Status: UP")

            sys.exit(0)

        else:

            print(f"[{timestamp}] Service Status: DOWN (Status: {response.status_code})")

            sys.exit(1)
```

```
    except requests.exceptions.RequestException as e:
        print(f"[{timestamp}] Service Status: DOWN")
        print(f"[{timestamp}] Error: {e}")
        sys.exit(1)


if __name__ == "__main__":
    check_service()
```

Save the file and exit nano by pressing **Ctrl+X**, then **Y**, and Enter.

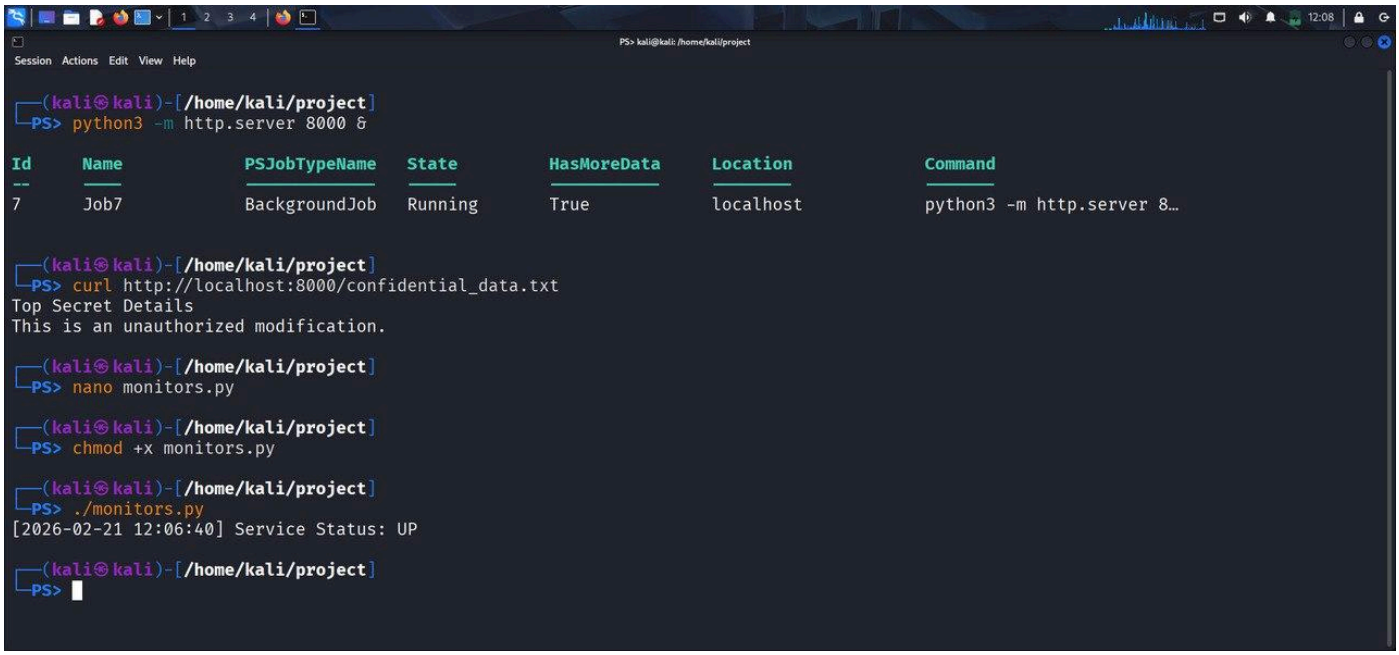
Make the script executable using **chmod**:

To ensure availability, let us use a script to check the status automatically.

Terminal:

```
┌──(kali㉿kali)-[~/project]
└─$ nano monitors.py
┌──(kali㉿kali)-[~/project]
└─$ chmod +x monitors.py
┌──(kali㉿kali)-[~/project]
└─$ ./monitors.py
Service Status: UP
```

- **`nano monitors.sh`**: Opens a text editor to write a script that checks if the server returns a "200 OK" status.
- **`chmod +x`**: Adds **eXecute** permissions to the script so it can be run as a program.



Summary of CIA in Kali Linux

Through these steps, I have applied the core tenets of information security:

- **Confidentiality**: Restricted access using `chmod 600`.
- **Integrity**: Prevented edits with `chmod 400` and detected tampering with `sha256sum`.
- **Availability**: Managed service uptime and automated monitoring.

Principle	Tool/Method	Outcome
Confidentiality	`chmod 600`	Only authorized users can read the data.
Integrity	`sha256sum`	Any unauthorized changes are detected.
Availability	`python3 server`	Ensures the service is accessible and monitored.